

Ta3awen (تعاون) — Project Overview & Architecture

Android MVP · Flutter front-end + Firebase back-end · October 2026

1. What the project is

Ta3awen (“cooperation”) is an Algerian mutual-aid app for neighbor-to-neighbor sharing. Instead of discarding usable items, a person posts them for free, and people nearby who need them can find them. Users can also post *requests* when they need something themselves. Everything is organized by wilaya and commune so exchanges stay local.

Core user journey:

- Sign up / log in with email + password (Firebase Auth).
- Browse the home feed: newest first, server-side search, filters by category / wilaya / commune, endless pages via a More button.
- Create a listing: title, description, category, wilaya, commune, condition, phone, up to 5 photos (auto-resized and stored inline as base64).
- Other users open the listing and contact the owner directly (phone dialer / WhatsApp deep link) — there is deliberately no in-app chat in the MVP.
- Owner marks an item as donated; other users can report abusive listings; a small admin panel (users, listings, reports) is gated by an `admins/{uid}` document.

2. Technology stack

Layer	Choice	Why
App (front-end)	Flutter 3 / Dart, Material 3, Arabic RTL (locale ar)	One codebase for Android now, iOS later
Auth (back-end)	Firebase Authentication — email/password	No server to run, SDK handles tokens/session
Database (back-end)	Cloud Firestore (project ta3awen-mvp-5a5ea)	Document store with live queries, rules and indexes
Images	Inline base64 inside Firestore documents	Skips Firebase Storage on the free tier; rules cap each image < 900 KB
Server code	None — Firestore security rules + composite indexes only	Cheapest possible MVP (Spark free tier)
Delivery	Signed release APK published on GitHub Releases	Direct download link, no Play Store yet

3. Front-end structure (Flutter)

All app code lives under `lib/`. Three hard rules keep it readable: screens render UI only, services own every Firebase call, models are plain data.

```

lib/
|- main.dart                boot: Firebase.initializeApp(), MaterialApp (ar RTL), home = AuthGate
|- firebase_options.dart    GENERATED per-platform keys (API key, project id) <- the link to Firebase
|- screens/
|   |- auth_gate.dart       watches FirebaseAuth.authStateChanges() -> LoginScreen or HomeShell
|   |- auth/                login_screen.dart, register_screen.dart (incl. forgot-password)
|   |- home/                home_shell.dart = 3 tabs (Home / My listings / Profile) + FAB
|   |                       feed_screen.dart = search box, filters, live head page + More pages
|   |                       listing_details_screen.dart, listing_form_screen.dart (create/edit)
|   |                       the owner's own ads (edit / delete / mark donated)
|   |                       user profile (name, wilaya, phone) + logout
|   |^- admin/              admin_screen.dart: users, listings, reports moderation
|- services/                ALL Firestore / Auth access lives here (UI never touches the DB)
|   |- auth_service.dart    signUp / signIn / signOut / sendPasswordReset
|   |- listing_service.dart FeedQuery, feedHead (live), feedMore (cursor pages), create/update/delete
|   |- image_service.dart   pick -> resize/compress -> base64 strings
|   |- user_service.dart    user profile document handling
|   |- report_service.dart  create abuse reports
|   |^- admin_service.dart  moderation queries + admin check (admins/{uid})
|- models/                  listing.dart (+ keywords field), app_user.dart, report.dart
|- data/algeria_data.dart   58 wilayas with their communes (static reference data)
|- utils/                   search_keywords.dart (Arabic normalizer + keyword builder),
|                           errors.dart, formatters.dart, b64_cache.dart (LRU image cache)
|^- widgets/                listing_card, base64_image, wilaya_commune_fields, state_views

```

4. Back-end structure (Firebase)

There is no custom server. The “back-end” is Firebase Auth + Firestore, guarded by security rules and accelerated by composite indexes.

Firestore collections:

```

users/{uid}                profile: fullName, wilaya, commune, contactPhone, disabled flag
admins/{uid}               empty marker doc = user is an admin (written only from console/SDK)
reports/{id}               reporterId, listingId, reason (readable only by admins)
listings/{id}              title, description, category, wilaya, commune, condition, status,
                           contactPhone, ownerId/ownerName, imageCount, image0 (thumb, base64),
                           keywords[] (denormalized for search), createdAt, updatedAt
                           ^- images/{imgId}  index (0..4), data (full photo, base64, < 900 KB)

```

Security rules (firebase.rules) — enforced server-side, not in the app:

- Reads require a signed-in user; admins and reports are admin-only.
- A listing can be created only by its owner, must pass `validListing()` (length limits, phone 9–20 chars, status must be “available”), and its `ownerId` can never be forged.
- Updates/deletes: owner or admin only; users can edit only safe profile fields; disabled accounts cannot write anything.
- Image documents: only the parent listing's owner, payload < 900 KB, index 0–4.

Indexes (firebase.indexes.json, 12 composite):

Every feed/search/filter combination the app can build is pre-declared: `status + createdAt DESC` (the base feed), plus `search (keywords array-contains-any)` crossed with `category / wilaya / commune / status`, ordered by `createdAt DESC, __name__ DESC` so cursor pagination works. Deployed once with `firebase deploy --only firestore:indexes`.

5. How the front-end links to the back-end



The mechanics, step by step:

- **Init.** `main.dart` calls `Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform)` — `firebase_options.dart` (generated by FlutterFire) carries the API key and project id, so every plugin knows *which* Firebase project to talk to. That file is the literal front-to-back link.
- **Auth.** `auth_service.dart` uses the `firebase_auth` plugin; after sign-in the SDK keeps a fresh ID token and attaches it to every Firestore request automatically. `AuthGate` just reacts to `authStateChanges()` and swaps Login/Home screens.
- **Data.** Screens call service methods only, e.g. `listingService.feedHead(query)`. The `cloud_firestore` plugin opens a persistent encrypted channel to Firestore; each request carries the ID token, Firestore evaluates `firestore.rules` server-side, and only then answers. Security is never trusted to app code.
- **Search & pages.** On write, the app denormalizes a `keywords[]` array (Arabic-normalized words + prefixes) onto the listing. The feed asks Firestore for `array-contains-any` + a status filter (only items whose status is the Arabic word for available) sorted by `createdAt DESC`; the first 20 docs stream live, older pages arrive with a cursor (`startAfter`) when the user taps More. The 12 composite indexes make each of those queries servable.
- **Write example.** Create listing -> `ImageService.process()` resizes photos to base64 -> `listing_service.create()` batches the listing doc + image docs in one write -> rules validate the shape -> other users' open feeds receive the new document through their live listener without refreshing.

6. Where the project stands

- MVP feature-complete and shipped as a signed APK (GitHub release, direct link); search, filters, pagination and rules/indexes verified end-to-end against the live project.
- Live data: 2 real listings + 100 seeded test listings (owner seed - test - owner - 001, wipeable with one script); 102 docs total, 101 available.
- Free-tier watch: Spark quotas (50K reads / 20K writes per day) are far above current traffic; billing + budget alert recommended before public launch.
- Before the Play Store: privacy policy + signup consent (Loi 18-07 / ANPDP declaration), keystore backup, store listing. Password-reset emails now arrive as “Ta3awen” in Arabic; custom subject/body awaits Google lifting its temporary template restriction.